01

클로저(Closure)

01. Closure

- 자신을 포함하고 있는 외부함수보다 내부함수가 더 오래 유지되는 경우, 외부 함수 밖에서 내부함수가 호출되더라도 외부함수의 지역 변수에 접근할 수 있는데 이러한 함수를 클로저(Closure)라고 부른다.
- 일반 내부 함수

```
function outerFunc() {
    let x = 10;
    let innerFunc = function () {
        console.log(x);
        };
    innerFunc();
}
outerFunc(); // 10
```

01. Closure

■ 클로저(Closure)

```
function outerFunc() {
    let x = 10;
    let innerFunc = function () {
        console.log(x);
        };
    return innerFunc;
}
let inner=outerFunc();
inner();
```

- 외부함수가 내부함수를 리턴함으로서 내부함수가 외부함수보다 오래 존재
- 내부함수가 외부함수의 지역변수를 접근해서 캡쳐
- 캡쳐된 지역변수는 외부함수가 소멸해도 계속 존재

01. Closure

```
function makeCounter(predicate) {
    let counter = 0;
    // 클로저를 반환
    return function () {
      counter = predicate(counter);
      return counter;
    };
  }
  function increase(n) {
    return ++n;
  }
  function decrease(n) {
    return --n;
  }
  const increaser = makeCounter(increase);
  console.log(increaser()); // 1
  console.log(increaser()); // 2
  const decreaser = makeCounter(decrease);
  console.log(decreaser()); // -1
  console.log(decreaser()); // -2
```

01. Closure

■ 이벤트 핸들러는 클로저

```
function ClickLogger() {
  const message = "This is a local variable";  // 지역 변수

  function handleClick() {
    console.log(message);  // 이벤트 핸들러가 지역 변수를 참조
  }

  return (
    <div>
      <button onClick={handleClick}>Log Message</button>
    </div>
  );
}
```

02

컴포넌트 전용 .css

01. CSS 모듈(CSS Modules)

- CSS 파일의 이름을 ComponentName.module.css로 작성
- 컴포넌트에서 import하고, 클래스를 객체처럼 사용

```
/* Button.module.css */
.button {
  background-color: blue;
  color: white;
  padding: 10px;
  border: none;
  border-radius: 5px;
  cursor: pointer;
}
```

01. CSS 모듈(CSS Modules)

```
// Button.js
import styles from './Button.module.css';
// CSS 모듈을 불러오기

function Button() {
  return <button className={styles.button}>Click
Me</button>;
}

export default Button;
```

03

useRef() HOOK

01. useRef

- DOM 요소에 직접 접근하거나 값을 유지하는 데 사용
- 렌더링 주기에 영향을 미치지 않으며, 값이 변경되어도 컴포넌트가 재렌더링되지 않음

```
import { useRef } from 'react';
function TextInput() {
  const inputEl = useRef(null);
  const focusInput = () => {
    inputEl.current.focus();
   };
  return (
    <div>
      <input ref={inputEl} type="text" />
      <button onClick={focusInput}>Focus the
input</button>
    </div>
  );
}
export default TextInput;
```

01. useRef

- Ref객체는 current속성 하나만을 가지며 매핑되는 대상을 저장
- DOM 요소 직접 제어: 특정 DOM 요소에 접근하여 포커스 설정, 스크롤 제어, 애니메이션 실행 등의 작업을 할 때 사용.
- 값의 유지 및 추적: 컴포넌트가 다시 렌더링되더라도 유지해야 하는 값을 저장하고 추적.
- 상태 변경 없이 값 업데이트: 상태 변경이 아닌 단순한 값 업데이트를 수행하여 컴포넌트가 불필요하게 재렌더링되지 않도록 함.
- useRef 는 동기적으로 동작, 내용 변경시 바로 적용

01. useRef

```
import { useRef, useState } from 'react';

function ClickTracker() {
  const clickCount = useRef(0);

  const handleClick = () => {
     clickCount.current += 1;
  console.log(`Button clicked ${clickCount.current} times`);
  };

  return (
    <div>
     <button onClick={handleClick}>Click Me</button>
    </div>
  );
}

export default ClickTracker;
```

01. useRef

```
import { useRef } from 'react';

function RefExample() {
  const myRef = useRef(0);

  const handleClick = () => {
    console.log('Before:', myRef.current); // 이전 값 출력
    myRef.current += 1;                    // 값 즉시 변경
    console.log('After:', myRef.current);  // 변경된 값 출력
  };

  return <button onClick={handleClick}>Increase Ref</button>;
}

export default RefExample;
```

04

useEffect Hook

01. useEffect

- 사이드 이펙트(side effects)를 처리하는 데 사용
  - 컴포넌트의 렌더링 과정 외에 발생하는 작업
  - 데이터 가져오기(비동기 요청), DOM 조작, 구독(subscription), 타이머 설정
- useEffect()의 기본 구조
  - useEffect(사이드 이펙트 처리 함수, 의존성 배열)
  - useEffect(()=>{……}, [….])
  - 사이드 이펙트 처리 함수
    - React가 Dom을 업데이트한 이후에 실행
    - 의존성 배열의 내용에 따라 실행
    - 의존성 배열이 비어 있을 때: 컴포넌트 마운트시 한번만 실행
    - 의존성 배열에 특정값이 포함되었을 때 : 배열 안에 포함된 값이 변경될 때마다 실행
    - 의존성 배열을 생략했을 때 : 컴포넌트가 랜더링 될때 마다 실행

01. useEffect

■ 컴포넌트의 생명주기

- 마운트(mount)
  - 리액트가 해당 컴포넌트의 렌더링을 완료하고, 브라우저의 DOM에 반영되었을 때
- 업데이트(update)
  - 상태나 props의 변경이 발생할 때마다 다시 렌더링
- 언마운드(unmount)
  - 컴포넌트가 화면에서 사라지거나, 라우팅 등으로 다른 컴포넌트로 교체될때

01. useEffect

- **useEffect의 리턴값으로 정리(cleanup) 함수를 반환**
  - 컴포넌트가 언마운트되거나, useEffect가 다시 실행되기 직전에 실행
  - 구독 해제, 타이머 제거, 메모리 누수 방지 등의 역할

```
import { useEffect } from 'react';
function ExampleComponent() {
  useEffect(() => {
    // 이 안에 사이드 이펙트 코드 작성
    console.log('Component rendered or updated');

    return () => {
      // Cleanup 함수:
       console.log('Cleanup');
    };
  }, []);  // 의존성 배열 (Dependency array)

  return <div>Example Component</div>;
}
```

01. useEffect

```
import { useState, useEffect } from 'react';

function ExampleComponent() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log(`Count value updated to: ${count}`);
  }, [count]); // count가 변경될 때만 실행됨

  return (
    <div>
      <p>{count}</p>
      <button onClick={() => setCount(count + 1)}>
       Increase Count</button>
    </div>
  );
}

export default ExampleComponent;
```

01. useEffect

```
import { useEffect } from 'react';

function TimerComponent() {
  useEffect(() => {
    const timer = setInterval(() => {
      console.log('Timer tick');
    }, 1000);

    return () => {
      clearInterval(timer); // 타이머 해제 (정리 함수)
      console.log('Timer stopped');
    };
  }, []);  // 마운트 시 한 번 실행

  return <div>Timer is running</div>;
}

export default TimerComponent;
```

01. useEffect

```
import { useEffect } from 'react';

function ExampleComponent() {
  useEffect(() => {
    console.log('Component mounted');
    // 마운트 시 한 번만 실행

    return () => {
      console.log('Component will unmount');
       // 언마운트 시 실행
    };
  }, []);
 // 빈 배열: 마운트 시 한 번 실행, 언마운트 시 정리 함수 실행

  return <div>Example Component</div>;
}

export default ExampleComponent;
```

01. useEffect

```
import { useRef, useEffect } from 'react';

function AutoFocusInput() {
  const inputRef = useRef(null);
  useEffect(() => {
    // 컴포넌트가 마운트되었을 때 실행
    if (inputRef.current) {
      inputRef.current.focus(); }
  }, []); // 빈 배열: 마운트시에만 실행됨

  return (
    <div>
      <input ref={inputRef} type="text"
       placeholder="자동으로 포커스됩니다" />
    </div>
  );
}

export default AutoFocusInput;
```

01. useEffect

```
import { useState, useRef, useEffect } from 'react';

function CounterComponent() {
  const [count, setCount] = useState(0);
  const previousCountRef = useRef(0);
// 이전 count 값을 저장하는 useRef

  useEffect(() => {
console.log(`Current count: ${count}, Previous count:
${previousCountRef.current}`);
previousCountRef.current = count;
  }, [count]); // count가 변경될 때마다 실행

  return (
    <div>
      <p>Current Count: {count}</p>
      <button onClick={() => setCount(count +
1)}>Increase Count</button>
    </div>
  );}
```

01. useEffect

```
import { useState, useEffect } from 'react';
function TimerComponent() {
  const [count, setCount] = useState(0);
  useEffect(() => {
    const interval = setInterval(() => {
      console.log(`Count: ${count}`);
    }, 1000);

    return () => {
       clearInterval(interval); // 기존 타이머를 정리
       console.log('Cleanup function executed');
    };
  }, [count]); // count가 변경될 때마다 useEffect 실행

  return (
    <div>
      <p>Count: {count}</p>
      <button onClick={() => setCount(count+1)}>
       Increase Count</button>
    </div>
  );}
```